GT — Game Tracker

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	GT — Game Tracker — Gerenciador de Jogos e Sessões
Gerente de Projeto	Gabriel Grabowski Bosco — estudante responsável
Gerente/Responsável de GCS-DevOps	Gabriel Grabowski Bosco — estudante responsável
Fábrica de Software / Squad	Desenvolvimento individual — Game Tracker. Par de revisão com acesso de escrita ainda não designado.
Repositório principal	https://github.com/gabrielgbosco/game-tracker-flutter.git

Histórico de Revisões

Data	Versão	Descrição	Autor
10/08/2026	1.0	Adaptação do template ao projeto Game Tracker, desenvolvido em Flutter/Dart para a N1 — Faixa 8.	Gabriel Grabowski Bosco
28/10/2026	1.1	Adequação do plano à configuração vigente do repositório: publicação do PGC, proteção da main contra push direto e ambientes staging e production.	Gabriel Grabowski Bosco

Sumário

Sumário 2
1. Introdução 4
1.1 Finalidade 4
1.2 Escopo 4
1.3 Definições, Acrônimos e Abreviações 4
1.4 Referências 4
1.5 Visão Geral 5
2. Cultura DevOps e Papéis na Gerência de Configuração 5
3. Ferramentas, Ambientes e Infraestrutura 5
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração 7
4.1 Itens de Configuração 7
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 8
5. Integração Contínua (CI) 8
5.1 Etapas do Pipeline de CI 8
5.2 Política de Pull Request e Revisão de Código 8
5.3 Critérios de Qualidade (Quality Gate) 8
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 9
6.3 Rollback e Recuperação 9
7. Infraestrutura como Código (IaC) 9
7.1 Versionamento de Infraestrutura 9
7.2 Ambientes Efêmeros 9
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 10
9. Baselines e Releases 10
9.1 Geração Automática de Baseline 10
9.2 Notas de Versão (Release Notes) 10
10. Estimativa do Status de Configuração 10
10.1 Monitoramento e Observabilidade 10
10.2 Métricas DORA 10
10.3 Relatórios e Auditorias 11
11. Marcos 11

12. Treinamento e Recursos 11

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 11

14. Quadro-Resumo do Pipeline CI/CD 11

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado ao aplicativo Game Tracker, desenvolvido em Flutter e Dart. O plano abrange o código-fonte, os modelos de dados, as telas, os widgets reutilizáveis, os recursos visuais, o arquivo de dependências, a documentação do projeto e este próprio plano, versionado no repositório. O aplicativo utiliza dados locais em memória durante a execução e não possui backend, banco de dados, API externa nem autenticação. A gerência de configuração está aplicada no GitHub: repositório público, branch main protegida e ambientes de implantação staging e production.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

1.5 Visão Geral

No Game Tracker, a Gerência de Configuração está aplicada no repositório público gabrielgbosco/game-tracker-flutter. O controle identifica os itens de configuração, impede o push direto na branch main e registra os ambientes staging e production. Não há workflow de CI/CD em .github/workflows nem infraestrutura de aplicação em nuvem. A execução do aplicativo continua local, por meio do Flutter.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Estudante responsável: define o escopo, implementa as funcionalidades, organiza as entregas e valida os requisitos da atividade.
Engenheiro DevOps / Release Engineer	O estudante configura o repositório público, a proteção da branch main e os ambientes staging e production.
Desenvolvedor(a)	Implementa as telas, modelos, widgets, navegação, regras de alteração de horas/avaliação e gerenciamento do estado local.
SRE / Operações *Site Reliability Engineering*	Não há operação remota do aplicativo. O acompanhamento da configuração é feito pelas regras de proteção, pelos ambientes e pelo histórico do GitHub.
QA / Automação de Testes	Realiza validações manuais das funcionalidades obrigatórias e verifica o comportamento do aplicativo durante a execução.
Segurança (DevSecOps)	Não há pipeline de segurança. As dependências permanecem declaradas no pubspec.yaml.
Comitê de Controle de Mudanças (CCM)	Não há comitê formal. A main exige uma aprovação de outra conta, ainda não designada. Em production, o revisor obrigatório é o estudante responsável.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas e recursos identificados no repositório e na configuração do GitHub:

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git e GitHub	Repositório público https://github.com/gabrielgbosco/game-tracker-flutter.git, com a branch main protegida.
Linguagem e framework	Dart + Flutter	Desenvolvimento do aplicativo mobile e composição das telas/widgets.

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Editor/execução	Flutter SDK, IDE e emulador/dispositivo Android	Execução, depuração e validação local do aplicativo.
Gerenciador de dependências	pubspec.yaml / Flutter Pub	Declaração e instalação das dependências do projeto.
Gráficos	fl_chart ^0.68.0	Geração do gráfico de barras de horas jogadas por jogo.
Recursos visuais	Imagens locais em assets/images/	Exibição das capas ilustrativas dos seis jogos.
CI/CD	GitHub Environments; workflow não implementado	staging sem revisor obrigatório e production com revisor obrigatório. Não há arquivo de pipeline.
Infraestrutura / containers / segredos	Configuração do GitHub	Proteção da branch e ambientes de implantação. Sem containers, nuvem de aplicação ou segredos no repositório.
Proteção de branch	GitHub Branch Protection	Recusa o push direto na main (erro GH006), exige pull request e 1 aprovação, inclusive para administrador.
Ambientes	GitHub Environments	staging não exige revisor. production exige a revisão de gabrielgbosco, sem exceção para administrador.

3.2 Ambientes e Infraestrutura

A execução do aplicativo ocorre localmente, com o Flutter, em emulador Android ou dispositivo compatível. No GitHub existem dois ambientes de implantação. O ambiente staging não exige revisor. O ambiente production exige a revisão de Gabriel Grabowski Bosco (gabrielgbosco), e administradores não podem ignorar essa regra. Esses ambientes governam uma implantação no GitHub; não há servidor publicado do aplicativo.

Ambiente	Finalidade	Gatilho de promoção
Execução local	Execução e validação do aplicativo Flutter em emulador Android ou dispositivo compatível.	Execução de flutter run após instalação das dependências.
Homologação / Staging	Ambiente GitHub staging. Não executa nem publica o aplicativo e não exige revisor obrigatório.	Não há promoção automática. O ambiente fica disponível para uma implantação futura.
Produção	Ambiente GitHub production. Não executa nem publica o aplicativo. O revisor obrigatório é gabrielgbosco, e administradores não ignoram a proteção.	Aprovação manual desse revisor antes de uma implantação no ambiente.

3.3 Estratégia de Branching e Versionamento

A estratégia adotada é GitHub Flow, com a branch main protegida:

- main — branch protegida. O push direto é recusado, inclusive para o administrador do repositório. A integração ocorre somente por pull request.
- feature/<descrição-curta> ou docs/<descrição-curta> — branches de curta duração para cada mudança.
- release/x.y — não utilizada nesta versão. Uma linha de release só será aberta se for necessário estabilizar uma versão.
- hotfix/<descrição> — correção urgente, também integrada por pull request. Não há push direto na main.

Versionamento: a versão do aplicativo permanece a declarada no pubspec.yaml, em Versionamento Semântico (MAJOR.MINOR.PATCH). Os commits descrevem a mudança. Este plano tem versão própria, registrada no histórico de revisões. Não há geração automática de changelog, porque não existe pipeline de release.

4. Identificação da Configuração

4.1 Itens de Configuração

Os itens de configuração versionados no repositório principal são:

- Código-fonte da aplicação em lib/ e recursos visuais em assets/images/. Não há suíte de testes automatizados versionada.
- Este Plano de Gerenciamento de Configuração, em docs/gcs/, como item de configuração formal.
- README.md, com a estrutura do projeto e a referência ao plano, além de pubspec.yaml e analysis_options.yaml.
- Configuração do GitHub mantida nas configurações do repositório: proteção da branch main e ambientes staging e production. Segredos não são armazenados no repositório.
- Não fazem parte desta versão: definições de pipeline, Dockerfiles, scripts de infraestrutura como código e migrações de banco de dados.
- Evidência da recusa do push direto na main, no arquivo docs/gcs/evidencia-recusa-push-direto-main.txt, registrada no pull request #2. Esse registro ainda não está na main, porque a proteção exige aprovação de outra conta.
- Documentação técnica do aplicativo, concentrada no README.md.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Arquivo Dart	snake_case para arquivos Dart	game_detail_screen.dart
Classe Dart	PascalCase	GameDetailScreen
Variável/método	camelCase	hoursPlayed / addHours()
Versão do projeto	Versão declarada no pubspec.yaml	1.0.0+1
Dependência	Nome e versão no pubspec.yaml	fl_chart: ^0.68.0

4.3 Estrutura do Repositório

O repositório https://github.com/gabrielgbosco/game-tracker-flutter está público, para permitir a verificação. A estrutura principal contém lib/main.dart, lib/theme/app_theme.dart, lib/models/game.dart, lib/data/game_repository.dart, lib/screens/, lib/widgets/, assets/images/, pubspec.yaml, analysis_options.yaml, README.md e docs/gcs/, onde está este plano. Não há diretório .github/workflows nem diretório de infraestrutura como código.

5. Integração Contínua (CI)

5.1 Etapas do Pipeline de CI

1. O repositório não contém .github/workflows. Build, lint, testes, análise de segurança e publicação de artefato não são executados automaticamente.
2. O controle vigente na integração é a proteção da branch main no GitHub.
3. O push direto para main é recusado pelo GitHub com o erro GH006 (Changes must be made through a pull request), inclusive para o administrador.
4. A mudança entra por uma branch auxiliar e por pull request.
5. O pull request exige 1 aprovação de outra conta com acesso de escrita.
6. A validação funcional permanece local, com flutter analyze, quando executado, e flutter run.
7. Não há quality gate de pipeline verde configurado como status check obrigatório.

5.2 Política de Pull Request e Revisão de Código

- Todo pull request para main exige, no mínimo, 1 aprovação de outra conta. O autor não aprova o próprio pull request.
- A conta do par de revisão com acesso de escrita ainda não foi adicionada. A regra permanece ativa e o merge fica pendente dessa aprovação.
- Não há pipeline de CI obrigatório. O merge não depende de um status check automático.

5.3 Critérios de Qualidade (Quality Gate)

O critério efetivamente imposto pelo GitHub é a aprovação do pull request. A verificação funcional, feita localmente, continua sendo: projeto compilável no Flutter; execução sem erros bloqueantes; dependências declaradas no pubspec.yaml; lint conforme flutter_lints, quando executado; navegação entre as telas; atualização das horas e das avaliações; horas não negativas; avaliações limitadas entre 0 e 10; funcionamento do gráfico e do sorteio aleatório.

6. Entrega e Implantação Contínua (CD)

Não há pipeline de CD nem publicação do aplicativo em servidor. Os ambientes staging e production existem no GitHub para governar uma futura implantação. A promoção a production exige a revisão de gabrielgbosco, e administradores não podem ignorar essa regra.

6.1 Etapas do Pipeline de CD

8. O ambiente staging está criado e não exige revisor.
9. Não há testes automatizados de aceitação ou de ponta a ponta em staging.

10. O ambiente production exige aprovação manual de Gabriel Grabowski Bosco (gabrielgbosco) antes de uma implantação nesse ambiente.
11. Não há deploy remoto do aplicativo. A execução atual continua local.
12. Não há teste de fumaça nem verificação de saúde remotos.
13. Não há geração automática de tag ou de changelog. A baseline observada é o commit da branch main depois do merge do pull request.

6.2 Estratégias de Implantação

Estratégia	Descrição
Implantação local	Execução do aplicativo em emulador ou dispositivo Android para validação manual.
Rolling update / Blue-Green / Canary	Não aplicável. Não há implantação remota do aplicativo. O ambiente production do GitHub é uma trava de aprovação, não um destino de deploy.
Feature Flags	Não identificadas no projeto.

6.3 Rollback e Recuperação

Não há rollback automatizado, porque o aplicativo não está implantado em servidor. A recuperação de uma mudança já integrada é um novo pull request que restaura o commit anterior da main. O histórico do Git é a cópia versionada usada nessa recuperação.

7. Infraestrutura como Código (IaC)

Não há infraestrutura de aplicação. O Game Tracker executa localmente, com dados em memória, sem backend, banco de dados, API externa ou serviço em nuvem. Os ambientes staging e production são configuração do GitHub, definida nas configurações do repositório, e não scripts de infraestrutura como código.

7.1 Versionamento de Infraestrutura

- Não se aplica o versionamento de infraestrutura por Terraform, Ansible, Helm ou Kubernetes.
- Não há arquivo de estado de infraestrutura. A configuração dos ambientes fica no GitHub, sob a conta gabrielgbosco.

7.2 Ambientes Efêmeros

Não há ambiente efêmero criado a cada pull request. A validação funcional é local. staging e production são ambientes permanentes do repositório.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

A mudança não é gravada por push direto na main. O fluxo é abrir uma branch de curta duração, registrar a alteração em um commit descritivo, validar o aplicativo localmente e abrir um pull request. O GitHub recusa o push direto. A alteração deste plano segue o mesmo fluxo, porque o PGC é item de configuração.

8.2 Comitê de Controle de Mudança (CCM)

Não há comitê formal com várias pessoas. A proteção da main exige uma aprovação além do autor, mas o par com acesso de escrita ainda não foi designado. A aprovação de implantação em production está atribuída a Gabriel Grabowski Bosco.

9. Baselines e Releases

9.1 Geração Automática de Baseline

A baseline do produto é o estado da branch main após o merge de um pull request. Este plano integra essa baseline em docs/gcs/. Não há tag automática nem imagem publicada.

9.2 Notas de Versão (Release Notes)

Não há geração automática de notas de versão. O registro da mudança é a descrição do pull request e o histórico de commits. A recusa do push direto está documentada no pull request #2.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O aplicativo não tem monitoramento remoto. Os ambientes staging e production não executam o aplicativo; eles registram a regra de promoção. A situação da configuração é verificada pelas regras de proteção da main, pelos ambientes do GitHub e pelo histórico de pull requests.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Não há deploy do aplicativo em produção. A frequência observável é a de merges na branch main.
Lead Time for Changes	Tempo entre o commit na branch auxiliar e o merge na main. Não há implantação em servidor.
Change Failure Rate	Não há implantações em produção para medir falha de deploy. O push direto recusado não altera a main.

Métrica	Descrição
Mean Time to Restore (MTTR)	Restauração por novo pull request que retorna a main ao commit anterior. Não há rollback automatizado.

10.3 Relatórios e Auditorias

A auditoria de GCS usa o histórico de commits e de pull requests, a regra de proteção da main e a configuração dos ambientes. Não há histórico de pipelines, porque o workflow automatizado não foi criado.

11. Marcos

Marcos do produto: organização do projeto Flutter e do pubspec.yaml; modelo Game e coleção inicial com seis jogos; telas de Dashboard, lista, detalhes, estatísticas e recomendação aleatória; estado local em memória; gráfico com fl_chart. Marcos de configuração desta versão: publicação do repositório público; inclusão deste plano em docs/gcs/; proteção da main contra push direto, com pull request e 1 aprovação; criação dos ambientes staging e production, o segundo com gabrielgbosco como revisor obrigatório.

12. Treinamento e Recursos

O projeto é desenvolvido individualmente. Os recursos necessários são Flutter SDK, Dart, editor ou IDE compatível, emulador ou dispositivo Android, Git e acesso à conta GitHub gabrielgbosco, responsável pelo repositório.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- A análise automática de vulnerabilidades em dependências não está integrada, porque não há pipeline de CI.
- Dependabot e Renovate não estão habilitados. Uma atualização de dependência, quando ocorrer, entra pelo mesmo fluxo de pull request.

Não há software desenvolvido por subcontratados ou fornecedores externos identificado no projeto. O único pacote externo relevante declarado é o fl_chart, utilizado para gerar o gráfico de barras na tela de Estatísticas. A dependência é declarada no pubspec.yaml e deve ser instalada por meio do gerenciador de dependências do Flutter. As imagens utilizadas são recursos locais gerados para o projeto.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo vigente, do commit à branch main. A promoção entre os ambientes do GitHub não publica o aplicativo.

Etapa	Descrição	Gatilho
Branch e validação local	A mudança nasce fora da main e é conferida com flutter pub get, flutter analyze e flutter run.	Início da alteração.
Pull request	A branch auxiliar é enviada ao GitHub e o pull request é aberto para a main. O push direto é recusado com o erro GH006.	Conclusão da validação local.
Revisão e merge	A proteção exige 1 aprovação de outra conta com acesso de escrita. Esse par ainda não foi designado, então o merge permanece pendente.	Abertura do pull request.
Baseline	O merge atualiza a main. O plano em docs/gcs/ acompanha o código como item de configuração.	Aprovação do pull request.
Ambientes	staging não exige revisor. production só aceita implantação depois da aprovação de gabrielgbosco. Nenhum dos dois publica o aplicativo.	Implantação futura direcionada ao ambiente do GitHub.